

Práctica 4: Localización con HMM

Un robot que no sabe dónde está

En esta práctica el robot no sabe en qué celda de la casa está. Mantiene una **creencia** $bel(x, y)$ — una distribución de probabilidad sobre las celdas libres — y la actualiza con los dos pasos de un **modelo oculto de Markov (HMM)**: un paso de **predicción** tras cada acción de movimiento, y un paso de **corrección** tras cada lectura de su sensor de contacto con paredes. A diferencia de la práctica del curso anterior, no hay balizas fijas: toda la información espacial viene de las paredes reales de la casa generada, y el movimiento es **dirigido** (una acción concreta, no un paseo aleatorio).

Se trabajará sobre el fichero `practica4_enunciado.py`, proporcionado parcialmente implementado. Se asume familiaridad con la API de `HouseWorld/Robot` (documentada en `practica0_intro.tex`).

1. Entrega

La entrega se realizará en el aula, durante la clase de prácticas del 22 de diciembre de 2026, a través de la tarea habilitada para ello en el aula virtual.

El día de la entrega, una vez entregada la práctica, se cumplimentará en el aula, en modo examen y sin acceso al código, un breve cuestionario en el que se pedirá explicar cómo se ha resuelto alguna cuestión concreta de la solución entregada (ver Sección 4 para un ejemplo del estilo de pregunta que cabe esperar).

2. Infraestructura proporcionada

Todo lo de esta sección ya está implementado en `practica4_enunciado.py`: no es TODO.

2.1. LocalizingRobot: constructor y consultas

<code>LocalizingRobot(world, *, move_success_prob=0.8)</code>	crea la creencia <code>self.belief</code> : un diccionario celda→probabilidad, uniforme sobre las celdas libres de <code>world</code> (nunca incluye celdas de muro)
<code>most_likely_pose()</code>	la celda con mayor creencia, y su probabilidad

2.2. transition_probability (para la Pregunta 6)

`transition_probability(x, y, x2, y2, action) -> float`

$P((x_2, y_2) \mid (x, y), action)$: exactamente el mismo modelo de movimiento que hay que implementar en la Pregunta 2, pero evaluado para una única pareja (origen, destino) en vez de para toda la creencia de golpe. Se da resuelto porque la Pregunta 6 (opcional) lo necesita evaluar celda a celda dentro de su algoritmo, y no tiene sentido pedir la misma física del movimiento dos veces.

2.3. Simulación para la demo

<code>simulate_directed_move(robot, action, rng, ...)</code>	ejecuta action sobre un Robot REAL con el modelo de movimiento dirigido (0.8 de éxito si la celda destino es libre; si no, o con probabilidad 0.2, no se mueve)
<code>simulate_noisy_wall_observation(true_pattern, rng)</code>	simula la lectura ruidosa del sensor a partir del patrón real, con el modelo de la Pregunta 3

Importante: estas dos funciones son las que **generan** el mundo real y sus lecturas ruidosas para la demo (usan **rng** para decidir qué pasa de verdad); el robot — y por tanto el código de las Preguntas 1 a 6 — nunca las llama ni sabe lo que deciden: solo ve la acción que *se intentó* y la lectura que *se midió*.

3. Desarrollo de la práctica

3.1. Paso 1: Ejecución inicial

Ejecuta el fichero tal cual se proporciona:

```
python3 practica4_enunciado.py
```

Debería generarse una casa y mostrarse la creencia inicial (uniforme) sobre el mapa, con un mapa de calor. Nada de lo que hay que implementar se llama todavía.

3.2. Paso 2: Pregunta 1 — `init_pose`

```
init_pose(x0, y0) -> None
```

Fija una creencia inicial **determinista**: $bel(x_0, y_0) = 1$, el resto de celdas a 0. Se usa para simplificar la demo (posición inicial conocida); el caso realista, con posición inicial desconocida, se puede probar con `--unknown-start`.

3.3. Paso 3: Pregunta 2 — `move` (predicción)

```
move(action) -> None
```

El paso de predicción del HMM:

$$bel_{t+1}(x_2, y_2) = \sum_{(x,y)} P((x_2, y_2) \mid (x, y), action) \cdot bel_t(x, y)$$

con el modelo de movimiento **dirigido**: $action \in \{N, S, E, W, STAY\}$.

- **STAY**: el robot se queda siempre donde está (determinista).
- **N/S/E/W**: si la celda destino es libre, se llega a ella con probabilidad `move_success_prob` (0.8 por defecto) y se permanece en el origen con probabilidad `1-move_success_prob`; si la celda destino es un **muro**, toda la masa se queda en el origen — el robot no puede "salir" de la casa intentándolo.

Comprueba que, tras cualquier secuencia de movimientos, la creencia sigue sumando 1 y nunca asigna probabilidad a una celda de muro.

3.4. Paso 4: Pregunta 3 — modelo del sensor

```
wall_pattern_at(x, y) -> Dict[str, bool]
sensor_likelihood(measured, true_pattern) -> float
```

`wall_pattern_at` devuelve el patrón de contacto con paredes **real** en (x, y) (las 4 direcciones, sin ruido) — el mismo cálculo que `Robot.wall_contact()`, pero evaluable en cualquier celda candidata, no solo en la posición real del robot.

`sensor_likelihood` es el modelo de ruido: $P(Z=\textit{measured} \mid \textit{true_pattern})$ vale 0,7 si coinciden en las 4 direcciones, 0,075 por cada uno de los 4 patrones que difieren en **exactamente 1** bit ($0,7 + 4 \times 0,075 = 1$), y 0 si difieren en 2 o más.

3.5. Paso 5: Pregunta 4 — observation (corrección)

```
observation(measured) -> None
```

El paso de corrección del HMM: multiplica cada $bel(x, y)$ por `sensor_likelihood(measured, wall_pattern_at(x, y))`, y normaliza para que la creencia vuelva a sumar 1.

3.6. Paso 6: Pregunta 5 — pipeline completo

No es un método nuevo: descomenta el resto del `main` y compruébalo tú mismo. Con la posición inicial conocida (`init_pose`, por defecto) y unos pocos pasos de movimiento+observación, la creencia debería concentrarse alrededor de la posición real del robot con confianza razonablemente alta — no total, porque tanto el movimiento (0.8) como el sensor (0.7) son ruidosos a propósito. Prueba también `--unknown-start`: sin posición inicial, la convergencia es más lenta y, en zonas muy simétricas de la casa (un pasillo recto interior, por ejemplo, donde muchas celdas comparten el mismo patrón "sin paredes cerca"), puede quedarse indecisa durante bastantes pasos — es una limitación real del sensor de contacto, no un fallo del código.

3.7. Paso 7 (opcional): Pregunta 6 — most_likely_trajectory

```
most_likely_trajectory(start_belief, actions, observations)
-> List[Coord]
```

Implementa el algoritmo de **Viterbi**: la secuencia de posiciones $x_0, \dots, x_T$ que **maximiza** $P(x_0, \dots, x_T \mid z_0, \dots, z_T, a_0, \dots, a_{T-1})$ — que **no** tiene por qué coincidir con encadenar el `most_likely_pose()` de cada paso de filtrado por separado (eso da la secuencia de estados individualmente más probables, que no tiene por qué ser una trayectoria consistente: podría "saltar" entre dos celdas que ni siquiera son alcanzables la una desde la otra en un paso). `len(observations)` debe ser `len(actions) + 1` (una observación inicial más una por cada acción).

Es programación dinámica hacia delante: para cada instante t y cada celda, guarda la probabilidad de la **mejor** trayectoria que termina ahí (δ) y de qué celda viene esa trayectoria (ψ , para poder reconstruir el camino al final recorriendo los punteros hacia atrás). Usa `transition_probability` (dado, Sección 2) y `sensor_likelihood/wall_pattern_at` (Pregunta 3) en cada paso.

4. Ejemplo ilustrativo de pregunta del cuestionario

Por ejemplo: cambiar el modelo de movimiento para que además pueda "resbalar" a una celda perpendicular a la dirección pedida con pequeña probabilidad (0.8 en la dirección pedida, 0.1 a cada uno de los dos lados perpendiculares, si esas celdas son libres), y pedir trazar a mano

el resultado de `move` para una secuencia corta de 2 o 3 acciones — mismo estilo que el Ejercicio 2 de la modificación de diciembre de 2025. Esto no es la pregunta real, solo un ejemplo del estilo esperado.

5. Entregables

- Un único fichero `practica4_enunciado.py` completado, renombrado a `practica4.py`, entregado en la tarea del aula virtual habilitada para ello.
- El código debe ser legible y estar debidamente tabulado. No es necesario comentarlo explicando su funcionalidad.